

FETT — Project Plan

Rafe Dunlop

Department of Computer Science and Software Engineering
University of Canterbury
Christchurch, New Zealand
rdu46@uclive.ac.nz

I. INTRODUCTION

Anaé de Baets introduces Fault Injection as an Educational and Training Tool in Cybersecurity (FETT) [1] to unify theoretical cybersecurity education with experiential learning. FETT offers injection of 22 á la carte cybersecurity faults based on the Open Worldwide Application Security Project (OWASP) Top 10:2021 [2] into Django web applications. By transforming preexisting applications, educators can use FETT to rapidly prototype cybersecurity training environments without additional development.

However, FETT is limited by its narrow input domain [1] and restriction to OWASP Top 10:2021 vulnerabilities: with the release of OWASP Top 10:2025 [3], some injections of the most relevant, contemporary web application vulnerabilities are not represented. Therefore, this project aims to introduce alternative, more robust, injection methods to reduce the fragility of injectors and implement new injectors corresponding to OWASP Top 10:2025 vulnerabilities. Where successful, these improvements increase the co-domain of relevant, vulnerable training environments FETT can synthesise. Further, compatibility with non-standard Django application structures makes outputs of FETT more representative of the population of deployed Django applications, especially more representative of vulnerable deployed applications [4].

It has been demonstrated that FETT can already produce vulnerable training environments tractably [1], so improving its utilisation of the plentiful existing (and growing) base of publicly available web applications [5] is viewed as the predominant next step to improve the holistic efficacy of FETT as a pedagogical tool for experiential learning.

In II-A the organisation and basis of the project is detailed with evaluation results of FETT and other current approaches, whereas II-B formalises the objectives of the current project. III includes a set of milestones (III-A) which dictate the project timeline, as well as a risk assessment and contingency plan (III-B). A task breakdown and cumulative plot of expected hour contributions are detailed in appendix A.

II. BACKGROUND AND OBJECTIVES

A. Background

FETT was developed by Baets for her master's thesis with the Université de Namur, supervised by Guillaume Nguyen, Xavier Devroey and Fabian Gilson (2023-2025). Moving forward, the project, to be undertaken by one student, is owned

and supervised by Gilson at The University of Canterbury (UC) in his capacity as Associate Professor in the Department of Computer Science and Software Engineering (CSSE) at UC.

The evaluation of FETT conducted by Baets [1] demonstrated that, for 6 selected top-performing student assignments, FETT had a successful injection rate of 70%, falling to 41% for the 5 open-source projects selected. A total successful injection rate of 57% was observed. Baets justifies that some injections were unsuccessful because sample applications were missing injection prerequisites, while others were unable to correctly traverse non-standard Django application structures.

DVWA [6], Juice Shop [7], WebGoat [8], VulnGen [9] and Hack The Box [10] all offer pedagogical tools for experiential cybersecurity learning with considerable relative merits [11]. However, none of these offers generation of vulnerable environments based on a prior application, so are necessarily limited in the quantity of distinct training material they can provide. VulnGen allows for the generation of virtual machines with specified faults, but these faults are applied in identical environments.

B. Objectives

The formal project objectives are given in the form of research questions (RQs). Definitions, success criteria and evaluation metrics follow. Any definitions specified apply to all RQs.

- **RQ1:** Which methods used in the prior implementation of FETT limit its scope and reliability?
- **RQ2:** What methods used in pattern matching, mutation testing or other fields are **a)** applicable to pedagogical fault injection in general and **b)** can redress identified limitations in FETT?
- **RQ3:** Which of the methods identified in **RQ2** can be leveraged to improve the scope, reliability and relevance of FETT?

RQ1 uses the release version evaluated in the published paper [1] as the definition of prior implementation. Scope is defined to be the domain size of valid target Django applications. Reliability, for each fault injector, is the proportion of injections that are successful, given that a valid injection opportunity exists, i.e. there exists functionality into which the fault can

be injected. A fault injection is successful when it introduces the correct fault without affecting application operability. **RQ1** can be deemed successful wherein it identifies assumptions, algorithms, dependencies or inconsistencies which narrow the scope or reliability of FETT itself or its individual injectors. This criterion can be evaluated upon the total number of failed injection cases from the published paper’s evaluation which cannot succeed because of one or more of the limitations identified.

RQ2 is successful if algorithms, frameworks or approaches utilised in literature are found which can be feasibly leveraged to redress the limitations of FETT, **a)** for fault injection generally, and **b)** which is applicable to FETT itself ($b \subseteq a$). This is quantifiable through the proportion of identified limitations of FETT which the best candidate solution or combination of systems, can feasibly redress, either within the existing system or through an alternative solution.

RQ3 defines the relevance of FETT to be its coverage of the most currently prominent web vulnerabilities as published in **OWASP Top 10:2025**. **RQ3** is successful if methods identified in **RQ2** can be demonstrated to redress limitations identified in **RQ1** and introduce vulnerability injectors for **OWASP Top 10:2025** vulnerabilities. This research question has two key evaluation metrics: the proportion of failed injection cases which succeed upon re-performing the evaluation from the published paper; the proportion of **OWASP Top 10:2025** vulnerabilities for which fault injectors are provided.

III. PLAN

A. Milestones

Project delivery is subdivided into 9 separate milestones in Table I below. Five (M_i^A) are academic; four are project-specific (M_i^P). Associated deliverables, D_i , are listed with milestones. Week refers to the expected UC calendar week of completion. Priorities and dependencies between deliverables are annotated by a code $I : D_1, \dots, D_n$ where I refers to the ascending priority level ($I - V$), and $D_1, \dots, D_n$ are the dependent deliverables. Dependencies are implicitly recursive.

B. Risk Analysis and Contingency Plan

The milestones specified indicate an ideal project timeline. However, some hazards introduce the possibility of alternative timelines or deliverable subsets being required. These are classified in Table II below with applicable mitigation strategies. $Risk = Impact \times Likelihood$ and L , I and R correspond to likelihood, impact and risk respectively. Both impact and likelihood are scaled from 1-3, corresponding to low/moderate/high impact/likelihood. Hazards are sorted in descending order on risk level and risks with a score of one were disregarded. Overall, exposure is low because the project is owned internally and no external factors impose hard deadlines on project deliverables. Dedicated contingency plans are offered for Risks of 6 or more.

If no applicable candidate solutions offer feasible solutions applicable to FETT itself after broadening the search to

TABLE I
PROJECT MILESTONES AND DELIVERABLES

	Milestone & Deliverable	Week
M_1^A	Project Plan — D_1 plan PDF ($II : \emptyset$)	5
M_1^P	Limitations Analysis — D_2 : report identifying causes of injection failure per injector with discussion and calculation of RQ1 evaluation metric ($II : D_1$)	10
M_2^A	Demonstration of interim Research Results — D_3 : presentation slides ($I : D_1$)	12
M_2^P	Candidate State-of-the-art Methods — D_4 : tabular report showing adherence to inclusion criteria of candidate methods for RQ2 part a ($III : D_2$) and D_5 : part b ($II : D_2$)	14
M_3^P	Implementation of Candidate Methods and OWASP Top 10:2025 injectors — D_6 : tagged commit in dedicated repository ($IV : D_4, D_5$)	19
M_4^P	Evaluation — D_7 : Tabular replication of the evaluation conducted in published paper with all injectors ($IV : D_4, D_5$)	20
M_3^A	Final Report — D_8 : report PDF ($V : D_6, D_7$)	23
M_4^A	Showcase Presentation — D_9 : presentation slides ($II : D_8$), D_{10} : showcase abstract ($I : D_7$)	24
M_5^A	Final Demonstration — D_{11} : presentation slides ($IV : D_8$)	26

TABLE II
PROJECT RISK ASSESSMENT

Hazard	L	I	R	Mitigation Strategy
No applicable state-of-the-art methods applicable to FETT exist for limitations identified	2	3	6	Broaden literature scope to adjacent fields, consider fault injection applications not applicable to FETT
State-of-the-art methods cannot be demonstrated to redress limitations	2	2	4	Require minimal proof-of-concept spike implementation to validate feasibility for success criteria of RQ2
Overfitting to limitations exposed by the original evaluation dataset	1	3	3	select additional projects to reserve as unseen data for validation, broaden literature review to fault injection applications in general
Replication of evaluation cannot be performed within time constraints	1	2	2	Limit evaluation to injection scenarios that have not already been successful
Dependencies used in selected candidate methods become unsupported or incompatible	1	2	2	include dependency maximum as an exclusion criteria for selection criteria of RQ2 candidate methods

adjacent fields, a contingency plan is offered: an alternative fault injection configurator will be delivered using methods identified. The project can still succeed at key objectives, broadening scope and reliability and introducing injection options for **OWASP Top 10:2025**.

REFERENCES

- [1] de Baets, A. D. Baets, G. Nguyen, X. Devroey, and F. Gilson, "Fett: Fault injection as an educational and training tool in cybersecurity," *IEEE*, 2025. [Online]. Available: <https://gitlab.com/fabgilson/django-vulnerability-injector>.
- [2] "Owasp top 10:2021." [Online]. Available: <https://owasp.org/Top10/2021/>
- [3] "Owasp top 10:2025." [Online]. Available: <https://owasp.org/Top10/2025/>
- [4] I. Chowdhury and M. Zulkernine, "Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities," *Journal of Systems Architecture*, vol. 57, pp. 294–313, 3 2011. [Online]. Available: https://doi.org/10.1007/3-540-44652-4_39
- [5] P. Pickerill, H. J. Jungen, M. Ochodek, M. Maćkowiak, and M. Staron, "Phantom: Curating github for engineered software projects using time-series clustering," *Empirical Software Engineering* 2020 25:4, vol. 25, pp. 2897–2929, 5 2020. [Online]. Available: <https://link.springer.com/article/10.1007/s10664-020-09825-8>
- [6] "Damn vulnerable web application (dvwa): 1.0.7." [Online]. Available: <https://www.vulnhub.com/entry/damn-vulnerable-web-application-dvwa-107,43/>
- [7] "Owasp juice shop." [Online]. Available: <https://owasp.org/www-project-juice-shop/>
- [8] "Owasp webgoat." [Online]. Available: <https://owasp.org/www-project-webgoat/>
- [9] W. H. Ang, H. Guo, and E. G. Chekole, "Vulngen: Vulnerable virtual machine generator," in *Proceedings of the 17th IEEE International Conference on Service Operations and Logistics, and Informatics, SOLI 2023*. Institute of Electrical and Electronics Engineers Inc., 2023.
- [10] "Hack the box." [Online]. Available: <https://www.hackthebox.com/>
- [11] P.-C. Lim, G.-W. A. Chieng, H.-C. Ling, and N. Jali, "Owasp a03 injection vulnerability in web application development," *Journal of Advanced Research in Applied Sciences and Engineering Technology Journal homepage*, vol. 57, pp. 107–116, 2026. [Online]. Available: https://semarakilmu.com.my/journals/index.php/applied_sciences_eng_tech/index

APPENDIX A PROJECT MANAGEMENT EVIDENCE

The following table decomposes the milestones for this project into tasks (T_i) with the associated milestone, expected number of hours to be spent on them (Exp), effective hours spent (Eff) and the weeks where work is expected to be logged against a task. It can be assumed that the hours for each task are expected to be distributed evenly across all weeks the task is worked on. The cumulative hours spent, both effective and expected, are projected in Figure 1.

TABLE III
TASK BREAKDOWN

Milestone	Task Description	Exp	Eff	Weeks
$T_1 : M_1^A$	Research project briefs, meet with project owners, select project	10	9	1-2
$T_2 : M_1^A$	Research state-of-the-art and theses around project brief, refine research questions and write project plan	40	36	3-5
$T_3 : M_1^P$	Debug failed injection scenarios from published evaluation to identify root causes	40	N/A	6-9
$T_4 : M_1^P$	Analyse failures to formalise limitations and include in final report	10	N/A	8-9
$T_5 : M_2^P$	Conduct literature search for candidate methods, refine inclusion and exclusion criteria, map methods to limitations identified	30	N/A	10-12
$T_6 : M_2^P$	Produce tabular comparison, select candidate methods, compute RQ2 evaluation metric, include in final report	10	N/A	13
$T_7 : M_2^A$	Prepare and refine interim research demonstration slides, deliver interim checkpoint	10	N/A	11-12
$T_8 : M_3^P$	Prototype and integrate selected methods into FETT injectors	40	N/A	13-18
$T_9 : M_3^P$	Implement injectors for OWASP Top 10:2025 vulnerabilities	30	N/A	13-18
$T_{10} : M_4^P$	Re-perform full evaluation with new code across all new and prior injectors, analyse results, tabulate, calculate evaluation metric for RQ3 and include in final report	20	N/A	19-20
$T_{11} : M_3^A$	Draft, refine and submit final report	30	N/A	21-23
$T_{12} : M_{4-5}^A$	Prepare, create materials (including showcase abstract) for, and deliver showcase and final demonstration	35	N/A	21-27

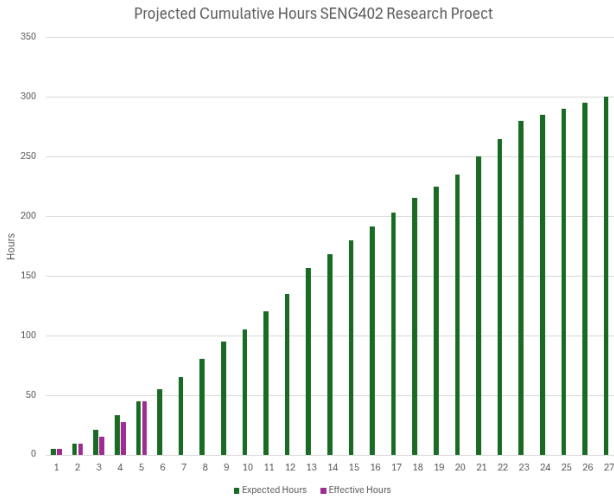


Fig. 1. Cumulative Expected and Effective Hours